

Form Builder

Descrição técnica do sistema, tecnologias utilizadas e decisões de arquitetura

Aplicação full stack para criação, publicação e resposta de formulários. O projeto atende ao desafio técnico de Full Stack Developer com backend Go independente, frontend React/TypeScript, persistência em PostgreSQL, autenticação por e-mail/senha e Google OAuth.

Item	Resumo
Backend	Serviço independente em Go 1.25+, responsável por API, regras de negócio, autenticação, validação, migrations e OpenAPI.
Frontend	SPA em React, TypeScript e Vite, com MUI, React Router e TanStack Query.
Banco de dados	PostgreSQL com tabelas relacionais para usuários, identidades, sessões, formulários, campos e respostas.
Docker	Docker Compose sobe apenas um container PostgreSQL local para facilitar o setup de desenvolvimento.
Autenticação	Login por e-mail/senha com bcrypt e sessão HTTP-only; login Google usando OAuth 2.0 Authorization Code no backend.

1. Visão Geral

O Form Builder permite que um administrador autenticado crie formulários, configure múltiplos campos, publique uma URL pública e consulte as respostas enviadas por visitantes. O visitante não precisa de login: basta acessar o link público do formulário publicado, preencher os campos e enviar a resposta.

A arquitetura separa claramente as responsabilidades. O frontend cuida da experiência de uso e consome a API. O backend Go concentra contratos, regras, validações e persistência. O banco PostgreSQL armazena os dados. Nenhuma funcionalidade de negócio depende de acesso direto do frontend ao banco.

- Usuário acessa a aplicação em `http://localhost:5173`.
- Realiza login por e-mail/senha, usuário padrão local ou Google OAuth.
- Acessa a tela de formulários salvos e, depois, a área administrativa.
- Cria e configura campos do formulário.
- Publica o formulário e recebe uma URL pública.
- Visitantes respondem pela URL pública sem autenticação.
- O administrador consulta, exporta ou exclui respostas recebidas.

2. Tecnologias Utilizadas

Tecnologia	Como foi utilizada
Go 1.25+	Implementa o backend como serviço independente. O comando principal aceita modos como <code>run</code> , <code>migrate up</code> , <code>migrate down</code> e <code>openapi</code> .
net/http	Fornecer o servidor HTTP e as rotas REST do backend sem substituir a responsabilidade da API por outro framework.
pgx/v5	Realiza a conexão com PostgreSQL e executa queries nos repositórios.
bcrypt	Gera e compara hashes de senha para autenticação por e-mail/senha.
PostgreSQL	Persiste usuários, identidades OAuth, sessões, formulários, campos, respostas e metadados de privacidade.
React 18	Constrói a interface de login, área administrativa, formulários salvos, respostas e formulário público.
TypeScript	Tipa os dados do frontend, principalmente a partir dos schemas gerados pelo OpenAPI.
Vite	Executa o ambiente de desenvolvimento e o build do frontend.
MUI	Fornecer componentes visuais, tema, botões, campos, tabelas, alerts, chips e layout responsivo.
React Router	Define as rotas públicas e protegidas do frontend.
TanStack Query	Gerencia chamadas à API, cache, loading, erros e invalidação de dados após mutations.
OpenAPI 3	Documenta o contrato da API em <code>backend/openapi/openapi.json</code> .
openapi-typescript	Gera os tipos TypeScript usados pelo frontend a partir do OpenAPI.

Tecnologia	Como foi utilizada
Docker Compose	Sobe um PostgreSQL 16 Alpine local com volume persistente e healthcheck.

3. Backend Go

O backend está em backend/ e é a fonte das regras de negócio. Ele expõe endpoints REST para autenticação, formulários, publicação e respostas. Também executa migrations e gera a especificação OpenAPI. A aplicação roda como um serviço independente com o comando `go run ./cmd/server run`.

- `cmd/server` contém a entrada da aplicação e os comandos `run`, `migrate` e `openapi`.
- `internal/auth` contém autenticação, sessões, OAuth Google e repositório de usuários.
- `internal/forms` contém regras e persistência de formulários e campos.
- `internal/responses` contém submissão pública, validação e consulta/exportação de respostas.
- `internal/database` contém conexão e runner de migrations.
- `internal/openapi` gera a especificação OpenAPI 3.

As variáveis de ambiente ficam no `.env` da raiz. O backend lê `ADDRESS`, `DATABASE_URL`, `FRONTEND_URL`, `SESSION_SECRET`, `COOKIE_SECURE`, credenciais Google e configuração do usuário administrador padrão.

4. Persistência e Migrations

O banco escolhido foi PostgreSQL por ser multiplataforma, robusto e adequado para dados relacionais. As migrations SQL ficam em `backend/migrations` e são embutidas no binário Go, o que permite executar `go run ./cmd/server migrate up` sem depender de uma CLI externa de migrations.

Tabela	Responsabilidade
users	Usuários administradores com e-mail, nome e hash de senha.
auth_identities	Vínculo entre usuário local e provedor externo, como Google.
sessions	Sessões com hash HMAC do token, expiração e revogação.
forms	Formulários, dono, status draft/published, slug público e metadados de privacidade.
form_fields	Campos configuráveis com tipo, posição, obrigatoriedade, opções e config.
form_responses	Respostas públicas, answers em JSONB, data de envio e ciência LGPD.

5. Docker e Container PostgreSQL

O Docker não substitui o backend Go nem o frontend React. Ele é usado como conveniência de desenvolvimento para subir somente o banco PostgreSQL. Isso reduz a fricção para quem clona o projeto, porque não é preciso instalar e configurar manualmente um banco local antes de testar a aplicação.

Configuração	Valor utilizado
Imagem	postgres:16-alpine
Serviço	postgres
Container	form_builder_postgres
Banco	form_builder
Usuário	form_builder
Senha	form_builder
Porta	5432:5432
Volume	postgres_data:/var/lib/postgresql/data
Healthcheck	pg_isready -U form_builder -d form_builder

O fluxo local com Docker é: `docker compose up -d postgres`, `go run ./cmd/server migrate up`, `go run ./cmd/server run` e `npm run dev` no frontend.

6. Autenticação por E-mail e Senha

A autenticação por e-mail e senha permite cadastro, login, logout e consulta do usuário autenticado. A senha nunca é salva em texto puro: o backend gera um hash com bcrypt e compara esse hash no login.

- POST /api/auth/register cria usuário local.
- POST /api/auth/login valida e-mail/senha e cria sessão.
- POST /api/auth/logout revoga a sessão.
- GET /api/auth/me consulta o usuário autenticado.
- O cookie form_builder_session é HTTP-only.
- O banco armazena apenas o hash HMAC do token de sessão, não o token puro.

Para facilitar avaliação e clone local, o projeto pode criar um administrador padrão durante migrate up quando SEED_DEFAULT_ADMIN=true: admin@gmail.com com senha 12345678. Em produção, a orientação é trocar essa senha ou desativar o seed.

7. Autenticação com Google OAuth

O login com Google foi implementado pelo backend usando OAuth 2.0 Authorization Code. O frontend não troca o code diretamente com o Google e não conhece o Client Secret. Ele apenas redireciona o usuário para GET /api/auth/google.

- O botão Continuar com Google chama a URL /api/auth/google do backend.
- O backend valida se GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET e GOOGLE_REDIRECT_URL estão configurados.
- O backend cria um state temporário e redireciona para a tela de consentimento do Google.
- O Google retorna para /api/auth/google/callback com code e state.
- O backend valida o state para reduzir risco de CSRF.
- O backend troca o code por token usando Client ID, Client Secret e Redirect URL.
- Com o access token, o backend consulta o perfil Google e exige e-mail verificado.
- Se a identidade Google já existir, usa o usuário vinculado.
- Se o e-mail já existir, vincula a identidade Google ao usuário existente.
- Se o e-mail não existir, cria um novo usuário e vincula a identidade.
- No fim, cria a mesma sessão HTTP-only usada pelo login por e-mail/senha e redireciona para /admin.

Cada pessoa que clonar o projeto precisa configurar seu próprio OAuth Client no Google Cloud, porque o Client Secret não deve ser versionado. Em ambiente local, o redirect autorizado é `http://localhost:8080/api/auth/google/callback` e a origem autorizada é `http://localhost:5173`.

8. Frontend React e Experiência Administrativa

O frontend está em frontend/ e usa React com TypeScript. A interface foi construída com MUI e organizada por features para separar telas, estilos e regras de interação. Os estilos MUI ficam em arquivos .styles.ts, mantendo lógica e estética separadas.

- A rota / exibe login e cadastro.
- A rota /admin exibe formulários salvos após autenticação.
- A rota /admin/workspace exibe a área administrativa para criar, editar, publicar e excluir formulários.
- A rota /admin/forms/:formId/responses exibe respostas do formulário.
- A rota /f/:slug abre o formulário público sem autenticação.
- A rota /privacidade exibe a política de privacidade.

Após login, o usuário vê primeiro a lista de formulários salvos. A área administrativa completa fica em menu protegido. Há botão de sair, botão para abrir o formulário público e botão compartilhar, que copia o link.

9. Formulários, Campos e Respostas

O administrador consegue criar formulários com múltiplos campos, salvar como rascunho, publicar e despublicar. Ao publicar, o backend gera um slug público e o frontend monta a URL /f/:slug.

Tipo de campo	Validação aplicada
Texto curto e texto longo	Campo obrigatório precisa ser preenchido quando marcado como obrigatório.
E-mail	Validação de formato de e-mail.
Número	Validação de valor numérico.
Telefone	Exige 12 dígitos numéricos no padrão definido para o projeto: 3 de DDD e 9 do telefone.
Seleção	A resposta precisa estar entre as opções configuradas.
Caixa de seleção	Aceita valor compatível com opção marcada/desmarcada.

As respostas públicas são enviadas para o backend, validadas de acordo com a definição publicada do formulário e persistidas. Na administração, as respostas podem ser visualizadas por formulário, exportadas em JSON, PDF e Excel, ou excluídas individualmente.

10. OpenAPI 3 e Client TypeScript Gerado

O contrato da API é documentado em OpenAPI 3. O backend possui comando próprio para gerar a especificação: `go run ./cmd/server openapi`. O arquivo gerado fica em `backend/openapi/openapi.json`.

O frontend não mantém manualmente os tipos e wrappers usados para consumir a API. O script `npm run generate:api` usa `openapi-typescript` para gerar `schema.ts` e um script local para gerar `client.ts`. O build do frontend também executa a geração antes do typecheck.

- `make openapi` gera `backend/openapi/openapi.json`.
- `make client` gera `frontend/src/api/generated/schema.ts` e `client.ts`.
- `npm run build` executa `generate:api`, `typecheck` e `vite build`.
- As features de auth, forms e responses derivam tipos do OpenAPI gerado.

11. LGPD e Privacidade

O sistema inclui um pacote mínimo de transparência e controle para apoiar a LGPD. Ele não substitui análise jurídica, mas implementa os recursos técnicos necessários para o fluxo do desafio.

- Página pública /privacidade com política de privacidade.
- Cada formulário possui e-mail do controlador, finalidade de tratamento e política de retenção.
- A publicação é bloqueada se os metadados de privacidade obrigatórios estiverem vazios.
- O formulário público exibe aviso de privacidade.
- O envio público exige confirmação de ciência.
- A resposta grava `privacyAcknowledgedAt`.
- O administrador pode exportar e excluir respostas.

12. Decisões de Arquitetura

- Monorepo para simplificar setup, revisão e execução local.
- Go no backend para atender ao requisito obrigatório e centralizar regras de negócio.
- Vite no frontend para manter uma SPA objetiva e deixar claro que a API pertence ao backend Go.
- PostgreSQL para persistência relacional, com JSONB onde a flexibilidade dos formulários é útil.
- Docker Compose apenas para o banco, evitando exigir instalação manual de PostgreSQL.
- OpenAPI como contrato entre backend e frontend, reduzindo divergência de tipos.
- Sessões com cookie HTTP-only e hash HMAC no banco para reduzir exposição do token.
- Separação de lógica e estética no frontend por features, app, api, lib e styles.

Limitação conhecida: o login Google local depende de credenciais OAuth próprias de quem clonar o projeto, porque o Client Secret não pode ser publicado no repositório. Essa limitação é intencional e documentada no README por segurança.

13. Execução Local Resumida

Etapa	Comando
Subir PostgreSQL com Docker	<code>docker compose up -d postgres</code>
Aplicar migrations	<code>cd backend; go run ./cmd/server migrate up</code>
Rodar backend	<code>cd backend; go run ./cmd/server run</code>
Instalar frontend	<code>cd frontend; npm install</code>
Rodar frontend	<code>cd frontend; npm run dev</code>
Abrir sistema	<code>http://localhost:5173</code>
Usuário padrão local	<code>admin@gmail.com / 12345678</code>

Com esses passos, a aplicação fica disponível localmente com API em `http://localhost:8080`, frontend em `http://localhost:5173` e PostgreSQL em `localhost:5432`.